

CS327: Compilers

Lecture 2

Shouvick Mondal

shouvick.mondal@iitgn.ac.in
January 2026

Compiler Writing

- is challenging.
- A compiler is a large program.
- A compiler must translate correctly potentially infinite set of programs that could be written in the source language.
- The problem of generating the optimal target code from a source program is undecidable.
 - Heuristics and Trade-offs.
- **Compilers is an area where Theory meets Practice.**

Static versus Dynamic

- Time
- Compilation
- Optimization
- Analysis
- Type
- Linking
- Scoping

Static versus Dynamic

- **Time:** compilation versus execution, preprocessor versus compilation
- **Compilation:** gcc versus jit
- **Optimization:** without and with input
- **Analysis:** without and with environment
- **Type**
 - strongly typed versus scripting languages
 - inheritance and virtual functions
- **Linking:** .a versus .so
- **Scoping**

Scoping: Static versus Dynamic

```
int i = 1;  
void f()  
{  
    printf("%d", i);  
}  
void main()  
{   int i = 2;  
    f();  
}
```

Static	Dynamic
1	2

Where do we use dynamic scoping?

Find the output of the program under static and dynamic scoping

```
int a = 1, b = 2, y = 3;
void gun(int x, int b) {
    printf("%d %d\n", x, b);
}
void fun(int x, int y) {
    printf("%d %d\n", x, y);
    gun(a, y);
}
void main() {
    int a = 3;
    {
        int b = 4;
        fun(a, b);
    }
    gun(a, b);
    fun(a, b);
}
```

Parameter Passing

- Call by value
 - This happens in C
- Call by reference
 - Supported in C++, aliasing
- Call by name
 - Macros
- Call by value-result
 - Supported in ADA

```
int i = 1;
int *ip = &i;
void f(int x) {
    int y;
    x = 3;
    ip = &y;
    x = i+x+2;
}
void main() {
    f(*ip);
    printf("%d", i);
}
```

Call by value: 1

Call by reference: 8

Call by name: 3

Call by value-result: 6

Classwork

Create an example that does not use pointers which produces different output under the four parameter passing schemes.

C program translated by gcc 9.4.0 to x86 assembly

```
#include<stdio.h>
int main()
{
    int a=1;
    printf("%d %d %d\n",++a,a++,++a);
    return(0);
}
```

4 2 4

```
main:
.LFB0:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $16, %rsp
movl $1, -4(%rbp)
addl $1, -4(%rbp)
movl -4(%rbp), %eax
leal 1(%rax), %edx
movl %edx, -4(%rbp)
addl $1, -4(%rbp)
movl -4(%rbp), %edx
movl -4(%rbp), %esi
movl %edx, %ecx
movl %eax, %edx
leaq .LC0(%rip), %rdi
movl $0, %eax
call printf@PLT
movl $0, %eax
leave
.cfi_def_cfa 7, 8
ret
.cfi_endproc
```

gcc -S -00

```
main:
.LFB23:
.cfi_startproc
endbr64
subq $8, %rsp
.cfi_def_cfa_offset 16
movl $4, %r8d
movl $2, %ecx
movl $4, %edx
leaq .LC0(%rip), %rsi
movl $1, %edi
movl $0, %eax
call __printf_chk@PLT
movl $0, %eax
addq $8, %rsp
.cfi_def_cfa_offset 8
ret
.cfi_endproc
```

gcc -S -01

```
main:
.LFB23:
.cfi_startproc
endbr64
subq $8, %rsp
.cfi_def_cfa_offset 16
movl $2, %ecx
movl $4, %edx
xorl %eax, %eax
movl $4, %r8d
leaq .LC0(%rip), %rsi
movl $1, %edi
call __printf_chk@PLT
xorl %eax, %eax
addq $8, %rsp
.cfi_def_cfa_offset 8
ret
.cfi_endproc
```

gcc -S -02

C program translated by clang 13.0.0 to x86 assembly

```
#include<stdio.h>
int main()
{
    int a=1;
    printf("%d %d %d\n",++a,a++,++a);
    return(0);
}
```

2 2 4

Why did the output
change?

```
main:
.cfi_startproc
# %bb.0:
    pushq   %rbp
.cfi_def_cfa_offset 16
.cfi_offset %rbp, -16
    movq    %rsp, %rbp
.cfi_def_cfa_register %rbp
    subq    $16, %rsp
    movl    $0, -4(%rbp)
    movl    $1, -8(%rbp)
    movl    -8(%rbp), %esi
    addl    $1, %esi
    movl    %esi, -8(%rbp)
    movl    -8(%rbp), %edx
    movl    %edx, %eax
    addl    $1, %eax
    movl    %eax, -8(%rbp)
    movl    -8(%rbp), %ecx
    addl    $1, %ecx
    movl    %ecx, -8(%rbp)
    movabsq $.L.str, %rdi
    movb    $0, %al
    callq   printf
    xorl    %eax, %eax
    addq    $16, %rsp
    popq    %rbp
.cfi_def_cfa %rsp, 8
    retq
.Lfunc_end0:
.size     main, .Lfunc_end0-main
.cfi_endproc
```

```
main:
.cfi_startproc
# %bb.0:
    pushq   %rax
.cfi_def_cfa_offset 16
    movl    $.L.str, %edi
    movl    $2, %esi
    movl    $2, %edx
    movl    $4, %ecx
    xorl    %eax, %eax
    callq   printf
    xorl    %eax, %eax
    popq    %rcx
.cfi_def_cfa_offset 8
    retq
.Lfunc_end0:
.size     main,
.Lfunc_end0-main
.cfi_endproc
```

clang -S -O1

```
main:
.cfi_startproc
# %bb.0:
    pushq   %rax
.cfi_def_cfa_offset 16
    movl    $.L.str, %edi
    movl    $2, %esi
    movl    $2, %edx
    movl    $4, %ecx
    xorl    %eax, %eax
    callq   printf
    xorl    %eax, %eax
    popq    %rcx
.cfi_def_cfa_offset 8
    retq
.Lfunc_end0:
.size     main,
.Lfunc_end0-main
.cfi_endproc
```

clang -S -O2

clang -S -O0

C program translated by **icc 2021.7.1** to x86 assembly

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a=1;
```

```
    printf("%d %d %d\n",++a,a++,++a);
```

```
    return(0);
```

```
}
```

2 2 4

Why did the output
change?

main:

```

    .cfi_startproc
..__tag_value_main.1:
..L2:
                                #3.1
    pushq   %rbp                #3.1
    .cfi_def_cfa_offset 16
    movq    %rsp, %rbp          #3.1
    .cfi_def_cfa 6, 16
    .cfi_offset 6, -16
    subq    $32, %rsp           #3.1
    movq    %rbx, -8(%rbp)       #3.1[spill]
    movl    $1, -32(%rbp)        #4.7
    movl    $1, %eax             #5.24
    addl    -32(%rbp), %eax       #5.24
    movl    %eax, -28(%rbp)       #5.24
    movl    -28(%rbp), %eax       #5.24
    movl    %eax, -32(%rbp)       #5.24
    movl    -32(%rbp), %eax       #5.26
    movl    %eax, -24(%rbp)       #5.26
    movl    $1, %eax             #5.26
    addl    -32(%rbp), %eax       #5.26
    movl    %eax, -32(%rbp)       #5.26
    movl    $1, %eax             #5.32
    addl    -32(%rbp), %eax       #5.32
    movl    %eax, -20(%rbp)       #5.32
    movl    -20(%rbp), %eax       #5.32
    movl    %eax, -32(%rbp)       #5.32
    movl    $.L_2__STRING.0, %eax #5.2
    movl    -28(%rbp), %edx       #5.2
    movl    -24(%rbp), %ecx       #5.2
    .cfi_offset 3, -24
    movl    -20(%rbp), %ebx       #5.2
    movq    %rax, %rdi           #5.2
    movl    %edx, %esi           #5.2
    movl    %ecx, %edx           #5.2
    movl    %ebx, %ecx           #5.2
    movl    $0, %eax             #5.2
    ...
    .cfi_endproc

```

icc -S -00

main:

```

    .cfi_startproc
..__tag_value_main.1:
..L2:
                                #3.1
    pushq   %rsi                #3.1
    .cfi_def_cfa_offset 16
    pushq    $3                  #3.1
    popq     %rdi                #3.1
    xorl     %esi, %esi          #3.1
    call     __intel_new_feature_proc_init #3.1
                                # LOE rbx rbp r12 r13 r14 r15
..B1.5:
                                # Preds ..B1.1
                                # Execution count [1.00e+00]
    stmxcsr  (%rsp)              #3.1
    pushq    $2                  #5.2
    popq     %rsi                #5.2
    movl     $.L_2__STRING.0, %edi #5.2
    orl      $32832, (%rsp)       #3.1
    movl     %esi, %edx          #5.2
    pushq    $4                  #5.2
    popq     %rcx                #5.2
    xorl     %eax, %eax          #5.2
    ldmxcsr  (%rsp)              #3.1
..__tag_value_main.4:
#   printf(const char *__restrict, ...)
    call     printf              #5.2
..__tag_value_main.5:
                                # LOE rbx rbp r12 r13 r14 r15
..B1.2:
                                # Preds ..B1.5
                                # Execution count [1.00e+00]
    xorl     %eax, %eax          #6.8
    popq     %rcx                #6.8
    ....
    .cfi_endproc

```

icc -S -01

Server Explorer

- Data Connections
- Servers
- SharePoint Connections

```
Program.cs
ConsoleApp1
Program
Main(string[] args)

205
206
207
208
209
210
211

public static void Main(string[] args)
{
    int a = 1;
    Console.WriteLine($"{{++a}} {{a++}} {{++a}}");
    a = 1;
    Console.WriteLine("{0} {1} {2}", ++a, a++, ++a);
}
```

Solution Explorer

Search Solution Explorer (Ctrl+;)

- Solution 'proj1' (1 of 1 project)
- ConsoleApp1
 - Dependencies
 - Program.cs

Microsoft Visual Studio Debug Console

```
2 2 4
2 2 4
C#
C:\Users\shouvi
op\IITGN\playgr
j1\ConsoleApp1\
g\net9.0\Consol
```

```
C:\Users\shouvik\Desktop\IITGN\cache_test\call_conv.c - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

call_conv.c

1 #include<stdio.h>
2 int main()
3 {
4     int a=1;
5     printf("%d %d %d\n", ++a, a++, ++a);
6     return(0);
7 }
8
```

Developer Command Prompt for VS 2022

```
C:\Users\shouvik\Desktop\IITGN\cache_test>call_conv.exe
3 2 3
C/C++
C:\Users\shouvik\Desktop\IITGN\cache_test>
```

<https://github.com/SET-IITGN/CS-327-Compilers>

Google Chrome

Jan 4 5:53 PM

Inbox - shouvik.mondal

Inbox - shouvik.mondal

CS327_Lecture_1.pptx

SET-IITGN/CS-327-Compilers

github.com/SET-IITGN/CS-327-Compilers

SET-IITGN / CS-327-Compilers

Type to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

SET CS-327-Compilers Public

Edit Pins

Unwatch 1

Fork 0

Star 2

main 1 Branch 0 Tags

Go to file

Add file

<> Code

shouvik149 Update README to reflect additional language processor 6f1f0a4 · last week 27 Commits

Pro	Update ReadMe.md	last week
language_processor	Add files via upload	last week
mul_add_pred_swap	Clarify language specification statement in ReadMe	last week
playground	Update ReadMe.md	last week
yapl	Update ReadMe.md	last week
LICENSE	Initial commit	last week
README.md	Update README to reflect additional language processor	last week

README Apache-2.0 license

CS-327-Compilers

Jan-May 2026, IIT Gandhinagar

Getting Started

You can use the materials (codebase) in this repository to get started for building your own compiler. Four sample language processors (source code + binary) designed by me are available here.

- Pro (pro): Scanner, Parser, Analyzer of sentences expressed in Propositional Logic.
- language_processor (sm): Scanner, Parser, Analyzer of sentences expressing arithmetic operations in bash .

About

No description, website, or topics provided.

Readme

Apache-2.0 license

Activity

Custom properties

2 stars

1 watching

0 forks

Audit log

Report repository

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

C 98.9%

Other 1.1%

Suggested workflows

Based on your tech stack

14

Activities

Google Chrome

Jan 4 5:53 PM

Inbox - shouvick.mondal

Inbox - shouvick.mondal

CS327_Lecture_1.pptx

SET-IITGN/CS-327-Compi

github.com/SET-IITGN/CS-327-Compilers

README

Apache-2.0 license

Getting Started

You can use the materials (codebase) in this repository to get started for building your own compiler. Four sample language processors (source code + binary) designed by me are available here.

- Pro** (`pro`): Scanner, Parser, Analyzer of sentences expressed in Propositional Logic.
- language_processor** (`sm`): Scanner, Parser, Analyzer of sentences expressing arithmetic operations in `bash`.
- mul_add_pred_swap** (`maps`): Scanner, Parser, Analyzer of sentences expressing arithmetic operations with swapped precedence (+,*).
- yapl** (`yapl`): Scanner, Parser, Analyzer of programs written in a C-like language based on the ANSI C 2011 ISO C standard.

ANSI C 2011 ISO C standard

- [ANSI C grammar, Lex specification](#)
- [ANSI C Yacc grammar](#)

Lab Assignments will be linked, i.e., built upon the previous one, and should be performed in groups of size at most three. Each group will develop a compiler for a custom language of their choice, or a subset of any popular programming language. Compilation phases collectively implemented (spanning across all four lab assignments) need not be exhaustive.

Our Compilation Pipeline

(Character stream) --> **Front-end** --> Intermediate Representation (IR) --> **Back-end** --> (Target machine code)

Illustration

Obtained from [Rupesh Nasre](#).

```
z = x + y * 32 (Character stream)
```

Front-end

- Lexical Analyzer

```
<id,1> <=> <id,2> <+> <id,3> <*> <32>
```
- Syntax Analyzer

No packages published

[Publish your first package](#)

Languages

C 98.9%

Other 1.1%

Suggested workflows

Based on your tech stack

SLSA Generic generator
Generate SLSA3 provenance for your existing release workflows

Configure

CMake based, single-platform projects
Build and test a CMake based project on a single-platform.

Configure

MSBuild based projects
Build a MSBuild based project.

Configure

[More workflows](#)

[Dismiss suggestions](#)

15

[📖 README](#) [📄 Apache-2.0 license](#)

Our Compilation Pipeline

(Character stream) --> **Front-end** --> Intermediate Representation (IR) --> **Back-end** --> (Target machine code)

Illustration

Obtained from [Rupesh Nasre](#).

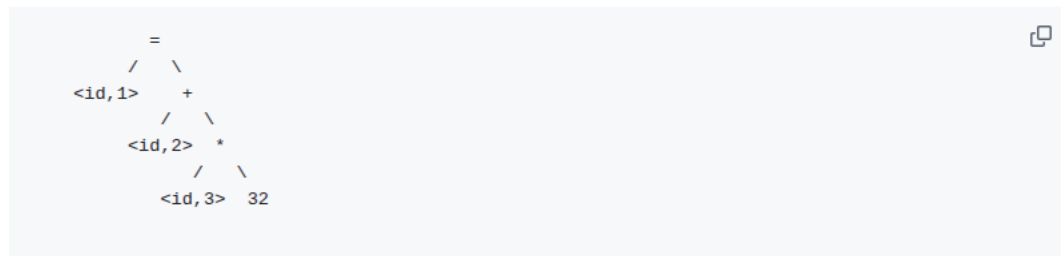
`z = x + y * 32` (Character stream)

Front-end

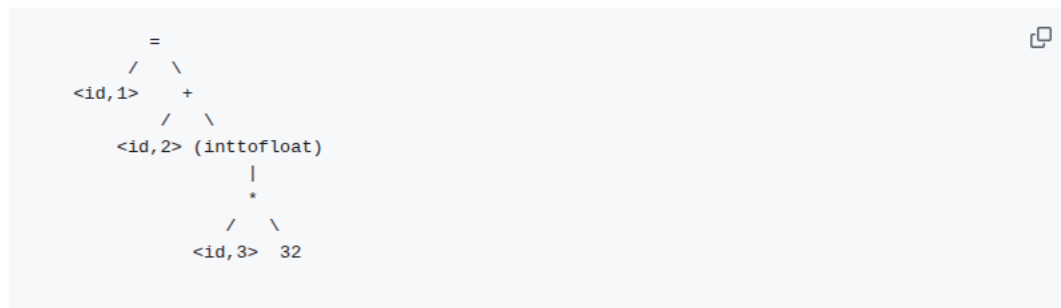
- Lexical Analyzer

$$\langle id, 1 \rangle \Rightarrow \langle id, 2 \rangle \langle + \rangle \langle id, 3 \rangle \langle * \rangle \langle 32 \rangle$$

- Syntax Analyzer



- Semantic Analyzer



- Intermediate Code Generator

Build and test a CMake based project on multiple platforms.

More workflows

Dismiss suggestions

ActivitiesGoogle ChromeJan 4 5:48 PM

Inbox - shouvick.mondalInbox - shouvick.mondalCS327_Lecture_1.pptxSET-IITGN/CS-327-Compi

github.com/SET-IITGN/CS-327-Compilers?tab=readme-ov-file#our-compilation-pipeline

READMEApache-2.0 license

- Intermediate Code Generator

```
t1 = id3 * 32
t2 = inttofloat(t1)
t3 = id2 + t2
id1 = t3
```
- Back-end

 - Machine-Independent Code Optimizer

```
t1 = id3 * 32
t2 = inttofloat(t1)
id1 = id2 + t2
```
 - Code Generator [AT&T syntax to be followed in gas](#)

```
LD R3, id3
MUL R3, R3, #32
ITOF R2, R3
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```
 - Machine-Dependent Code Optimizer [AT&T syntax to be followed in gas](#)

```
LD R3, id3
SHL R3, #5
ITOF R2, R3
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```

Software Tools to be used: [lex](#), [yacc](#), [gcc](#), [gas](#), and other relevant tools as and when necessary.

Texts/References

- Alfred Aho, Monica Lam, Ravi Sethi, Jeff Ullman, [Compilers: Principles, Techniques, and Tools](#), Pearson

Activities

Google Chrome

Jan 4 5:53 PM

Inbox - shouvik.mondal

Inbox - shouvik.mondal

CS327_Lecture_1.pptx

SET-IITGN/CS-327-Compilers

github.com/SET-IITGN/CS-327-Compilers

SET-IITGN / CS-327-Compilers

Type to search

<> Code Issues Pull requests Actions Projects Wiki Security Insights Settings

SET-IITGN

CS-327-Compilers

Public

Edit Pins Unwatch 1 Fork 0 Star 2

main 1 Branch 0 Tags

Go to file

Add file

<> Code

shouvik149 Update README to reflect additional language processor 6f1f0a4 · last week 27 Commits

Pro

language_processor

mul_add_pred_swap

playground

yapl

LICENSE

README.md

Update ReadMe.md

Add files via upload

Clarify language specification statement in ReadMe

Update ReadMe.md

Update ReadMe.md

Initial commit

Update README to reflect additional language processor

last week

last week

last week

last week

last week

last week

last week

README Apache-2.0 license

CS-327-Compilers

Jan-May 2026, IIT Gandhinagar

Getting Started

You can use the materials (codebase) in this repository to get started for building your own compiler. Four sample language processors (source code + binary) designed by me are available here.

Pro (pro)

: Scanner, Parser, Analyzer of sentences expressed in Propositional Logic.

language_processor (sm)

: Scanner, Parser, Analyzer of sentences expressing arithmetic operations in bash .

About

No description, website, or topics provided.

Readme Apache-2.0 license Activity Custom properties 2 stars 1 watching 0 forks Audit log Report repository

Releases

No releases published Create a new release

Packages

No packages published Publish your first package

Languages

C 98.9% Other 1.1%

Suggested workflows

Based on your tech stack

18

Files

main

Go to file

Pro

20_variables.prl

24_varialbes.prl

ReadMe.md

lex.yy.c

logic1.prl

logic10.prl

logic11.prl

logic12.prl

logic2.prl

logic3.prl

logic4.prl

logic5.prl

logic6.prl

logic7.prl

logic8.prl

logic9.prl

pro

pro.l

pro.y

y.tab.c

y.tab.h

language_processor

mul_add_pred_swap

playground

vanl

CS-327-Compilers / Pro /

↑ Top

ReadMe.md

A Syntactically Correct Sentence (well-formed formula or wff) in Propositional Logic

$\sim(P \rightarrow Q) \& ((Q \& P) \rightarrow Q)$

Analysis (output of the language processor: **ACCEPTED**)

Truth table for $\sim(P \rightarrow Q) \& ((Q \& P) \rightarrow Q)$

P	Q	$(P \rightarrow Q)$	$\sim((P \rightarrow Q))$	$(Q \& P)$	$((Q \& P) \rightarrow Q)$	$\sim(P \rightarrow Q) \& ((Q \& P) \rightarrow Q)$
F	F	T	F	F	T	F
F	T	T	F	F	T	F
T	F	F	T	F	T	T
T	T	T	F	T	T	F

The wff $\sim(P \rightarrow Q) \& ((Q \& P) \rightarrow Q)$ is...
Invalid
Consistent

A Syntactically Incorrect Sentence (ill-formed formula) in Propositional Logic

$(P \& Q)$

Analysis (output of the language processor: **REJECTED**)

Error class: syntax error

Error : error in line 1 caused by '(' [unmatched left parenthesis or it has invalid placement]

Error string: (P&Q

19

Activities

Google Chrome

Jan 4 5:53 PM

Inbox - shouvik.mondal

Inbox - shouvik.mondal

CS327_Lecture_1.pptx

SET-IITGN/CS-327-Compilers

github.com/SET-IITGN/CS-327-Compilers

SET-IITGN / CS-327-Compilers

Type to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

SET-IITGN

CS-327-Compilers

Public

Edit Pins

Unwatch 1

Fork 0

Star 2

main

1 Branch

0 Tags

Go to file

Add file

<> Code

shouvik149

Update README to reflect additional language processor

6f1f0a4 · last week

27 Commits

Pro

Update ReadMe.md

last week

language_processor

Add files via upload

last week

mul_add_pred_swap

Clarify language specification statement in ReadMe

last week

playground

Update ReadMe.md

last week

yapl

Update ReadMe.md

last week

LICENSE

Initial commit

last week

README.md

Update README to reflect additional language processor

last week

README

Apache-2.0 license

CS-327-Compilers

Jan-May 2026, IIT Gandhinagar

Getting Started

You can use the materials (codebase) in this repository to get started for building your own compiler. Four sample language processors (source code + binary) designed by me are available here.

- Pro (pro): Scanner, Parser, Analyzer of sentences expressed in Propositional Logic.
- language_processor (sm): Scanner, Parser, Analyzer of sentences expressing arithmetic operations in bash .

About

No description, website, or topics provided.

Readme

Apache-2.0 license

Activity

Custom properties

2 stars

1 watching

0 forks

Audit log

Report repository

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

C 98.9%

Other 1.1%

Suggested workflows

Based on your tech stack

20

1. Shell script involving arithmetic operations using the syntax of bash
2. Input will be in plain text(.txt) format and fed to the executable of my implementation [a.out]
3. " `display` " command[apart from bash feature] will be used for displaying result on screen. display is compulsory at the end of the script.
4. The only way of input is the `.txt` " `sm script` " and nothing in-between that is user will not be asked to enter a number, enter an array and similar waiting for user inputs.

Google Chrome

Jan 4 5:53 PM

Inbox - shouvik.mondal x

Inbox - shouvik.mondal x

CS327_Lecture_1.pptx x

SET-IITGN/CS-327-Compi x

+

github.com/SET-IITGN/CS-327-Compilers

🔍 Type to search

+

+

+

+

+

+

+

+

+

+

SET-IITGN / CS-327-Compilers

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

SET

CS-327-Compilers

Public

Edit Pins

Unwatch 1

Fork 0

Star 2

main 1 Branch 0 Tags

Go to file

Add file

<> Code

shouvik149 Update README to reflect additional language processor 6f1f0a4 · last week 27 Commits

Pro	Update ReadMe.md	last week
language_processor	Add files via upload	last week
mul_add_pred_swap	Clarify language specification statement in ReadMe	last week
playground	Update ReadMe.md	last week
yapl	Update ReadMe.md	last week
LICENSE	Initial commit	last week
README.md	Update README to reflect additional language processor	last week

README Apache-2.0 license

CS-327-Compilers

Jan-May 2026, IIT Gandhinagar

Getting Started

You can use the materials (codebase) in this repository to get started for building your own compiler. Four sample language processors (source code + binary) designed by me are available here.

Pro (pro)

: Scanner, Parser, Analyzer of sentences expressed in Propositional Logic.

language_processor (sm)

: Scanner, Parser, Analyzer of sentences expressing arithmetic operations in bash .

About

No description, website, or topics provided.

Readme

Apache-2.0 license

Activity

Custom properties

2 stars

1 watching

0 forks

Audit log

Report repository

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

C 98.9%

Other 1.1%

Suggested workflows

Based on your tech stack

 shouvick149 Update ReadMe.md

d96d07e · last week History

Name	Last commit message	Last commit date
..		
ReadMe.md	Update ReadMe.md	last week
a.out	Collections	last week
file.c	Collections	last week
file.s	Collections	last week

- > Pro
- > language_processor
- > mul_add_pred_swap
- ✓ playground
 - ReadMe.md
 - a.out
 - file.c
 - file.s
- > yapl
 - LICENSE
 - README.md

[ReadMe.md](#)

This is your playground!

```

/*
Filename: hello.s
example code written in the x86 assembly language (AT&T syntax)
*/

global:
    .string "hello%d\n"
    .globl main

global2:
    .string "%d\n"

main:
    push    %rbp
    mov %rsp, %rbp
    sub $16, %rsp
    mov $5, -12(%rbp)
    mov $6, -8(%rbp)
    mov $0, -4(%rbp)
    mov -12(%rbp), %edx
    mov -8(%rbp), %eax
    add %edx, %eax
    mov %eax, -4(%rbp)
    mov -4(%rbp), %eax
    mov %eax, %esi
    lea global2(%rip), %rax

```

Activities

Google Chrome

Jan 4 5:53 PM

Inbox - shouvick.mondal

Inbox - shouvick.mondal

CS327_Lecture_1.pptx

SET-IITGN/CS-327-Compilers

github.com/SET-IITGN/CS-327-Compilers

SET-IITGN / CS-327-Compilers

Type to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

SET-IITGN

CS-327-Compilers

Public

Edit Pins

Unwatch 1

Fork 0

Star 2

main

1 Branch

0 Tags

Go to file

Add file

<> Code

shouvick149

Update README to reflect additional language processor

6f1f0a4 · last week

27 Commits

Pro

Update ReadMe.md

last week

language_processor

Add files via upload

last week

mul_add_pred_swap

Clarify language specification statement in ReadMe

last week

playground

Update ReadMe.md

last week

yapl

Update ReadMe.md

last week

LICENSE

Initial commit

last week

README.md

Update README to reflect additional language processor

last week

README

Apache-2.0 license

CS-327-Compilers

Jan-May 2026, IIT Gandhinagar

Getting Started

You can use the materials (codebase) in this repository to get started for building your own compiler. Four sample language processors (source code + binary) designed by me are available here.

- Pro (pro)**: Scanner, Parser, Analyzer of sentences expressed in Propositional Logic.
- language_processor (sm)**: Scanner, Parser, Analyzer of sentences expressing arithmetic operations in `bash`.

About

No description, website, or topics provided.

Readme

Apache-2.0 license

Activity

Custom properties

2 stars

1 watching

0 forks

Audit log

Report repository

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

C 98.9%

Other 1.1%

Suggested workflows

Based on your tech stack

24

ActivitiesGoogle ChromeJan 4 5:50 PM

Inbox - shouvik.xInbox - shouvik.xCS327_Lecture_xSET-IITGN/CS-32_xCS-327-Compiler_xCS-327-Compiler_xCS-327-Compiler_xCS-327-Compiler_xCS-327-Compiler_x

github.com/SET-IITGN/CS-327-Compilers/tree/main/yapl

Filesmain+QGo to filet

Prolanguage_processormul_add_pred_swapplaygroundyapltestcasesMakefileReadMe.mdassign_op.cex1.cex2.clang_spec.odtlang_spec.pdflex.yy.cy.tab.cy.tab.hyaplyapliyaplyLICENSEREADME.md

CS-327-Compilers / yapl /

yaplyAdded scanner and parser yapl: a C-like language.last week

ReadMe.md

Scanning & Parsing for yapl : a C-like Language

The yapl specification is available [here](#).

yapl in action...

testcase_1.c

```
int *var=6;
struct mystruct *ms=&var;

int auto, static, inline;

extern void *k;
int p;
int p;
int p;

int *hh(char *p);

int main(int b)
{
    int auto=2,b=3,c;
    c=auto+b;
    printf("%d",c);

    struct player
    {
        int a;
        double c;
    };

    int *jj=&auto;

    System.out.print("java here");
    char echo[3]="bash here";
    myprintf("CS3300 here");
    printf(echo);
```

25

77.7 KB

CS 327: Compilers

January 2026

Scanning & Parsing for **yap_l**: a C-like Language

Task

Write input files for lex and yacc to generate lexical analyzer and parser for the following language description.

This language specification conforms to the ANSI C-2011 standard with the following requirements/modifications:

The language constructs:

✓ global variables, both normal as well as extern.

✓ function declaration, function definition, and function call (including recursion).

✓ for loop, while loop, do-while.

✓ if, if-else, if-else-if (nested/ladder) {if condition can be any expression including a literals/constants}.

✓ switch/case with case labeled with ':', must support fall-through, must support the optional default clause.

✓ variable declaration, definition, initialization locally/globally {multiple variables comma-separated}.

✓ strings (of the forms "... " (single), "... "..." (multiple)), character, integer and floating point literals.

✓ single (//) and multi-line (/*...*/) comments {multi-line comments should be properly closed lexically}.

✓ break, continue, and return statements.

✓ a statement must end in a semi-colon ';' like the usual standard of C.

The supported data types are:

✓ primary: int, long, short, float, double, void, char and their pointers.

✓ user-defined: structures (struct) {pointer dereferences must support arbitrary level of indirection.}

✓ derived: functions, arrays, and pointers: all supported data types. {no need to support function pointers.}

Supported operators (precedence and associativity same as in C):

✓ relational operators: <, >, ==, <=, >=, !=

✓ unary operators: +, -, !, *, &, ~

✓ binary operators: +, -, *, &, |, /, %, ^

✓ logical operator: &&, ||

✓ assignment operator: =

✓ suffix/postfix increment and decrement: ++, --

✓ structure field access operators: ->, .

✓ pointers: *, & {pointer dereferences must support arbitrary/multiple level of indirection, i.e., int *****p;}

✓ function call: () {any valid expression inside including blank, constants, and literals}

26

 Files

main

Go to file

- > Pro
- > language_processor
- > mul_add_pred_swap
- > playground
- ✓ yapl
 - > testcases
 - Makefile
 - ReadMe.md
 - assign_op.c
 - ex1.c
 - ex2.c
 - lang_spec.odt
 - lang_spec.pdf
 - lex.yy.c
 - y.tab.c
 - y.tab.h
 - yapl
 - yapl.l
 - yapl.y
 - LICENSE
 - README.md

CS-327-Compilers / yapl / lang_spec.pdf

77.7 KB

- ✓ primary: int, long, short, float, double, void, char and their pointers.
- ✓ user-defined: structures (struct) {pointer dereferences must support arbitrary level of indirection.}
- ✓ derived: functions, arrays, and pointers: all supported data types. {no need to support function pointers.}

- Supported operators (precedence and associativity same as in C):

- ✓ relational operators: <, >, ==, <=, >=, !=
- ✓ unary operators: +, -, !, *, &, ~
- ✓ binary operators: +, -, *, &, |, /, %, ^
- ✓ logical operator: &&, ||
- ✓ assignment operator: =
- ✓ suffix/postfix increment and decrement: ++, --
- ✓ structure field access operators: ->, .
- ✓ pointers: *, & {pointer dereferences must support arbitrary/multiple level of indirection, i.e., int *****p;}
- ✓ function call: () {any valid expression inside including blank, constants, and literals}
- ✓ array subscripting: [] {any valid expression inside including constants and literals}
- ✓ sizeof() {operands: <typename> or unary expression}
- ✓ typedef: (<typename>)

- Constructs **NOT** supported by this language:

- ```

x preprocessors: none allowed. {no #includes, #defines needed anywhere}
x ... (ellipsis), ?: (ternary operator) {function vararg list specification not required}
x operators: >>=, <<=, +=, -=, *=, /=, %=, &=, ^=, |=, <<, >>
x constructs/keywords: auto, const, goto, inline, register, restrict, signed, static, typedef, unsigned, enum,
union, volatile, _Alignas, _Alignof, _Atomic, _Bool, _Complex, _Generic, _Imaginary, _Noreturn,
_Static assert, Thread local, func

```

- The following operator is *introduced*:

| Operator | Description                                                                                                                                                                                                                                                          |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <=>      | <p><i>Three way comparison.</i></p> <p><math>a &lt;=&gt; b</math> returns the values -1, 0, or 1 depending on whether <math>a &lt; b</math>, <math>a == b</math>, or <math>a &gt; b</math></p> <p>Precedence and associativity same as: &lt;, &gt;, &lt;=, &gt;=</p> |

**Note:**



Files

main

Go to file

>  Pro

- language\_processor

```
> mul add pred swap
```

- >  playground

yapl

```
> testcases
```

Makefile

 ReadMe.md

 assign\_op.c

ex1.c

 ex2.c

 lang\_spec.odt

 lang\_spec.pdf

lex.yy.c

 y.tab.c

 y.tab.h

yapl

yapl.l

 yapl.y

 LICENSE

 README.md

CS-327-Compilers / yapl / lang\_spec.pdf

77.7 KB

- **#global\_declarations:** Total number of global declarations (entities) including functions, global variables, extern variables at the global scope. Note that declaration and definition for the same function are counted separately.
- **#function\_definitions:** Total number functions having bodies, i.e. defined.
- **#integer\_constants:** Total number of integer constants that appear *anywhere* throughout the program. Should support hexadecimal constants like 0x1234. Upper-case 'X' for hex need not be supported.
- **#pointers\_declarations:** Total count of pointers that are declared through the file. Note that if a function returning a pointer, has both prototype (declaration) and definition, they are counted separately. Pointer casts should not be counted.
- **#ifs\_without\_else:** Total count if statements that have no associated else clause.

- **if-else max-depth:** The max height of if-else-if ladder. The height is recursively defined as follows.

*height*(Ladder)

=0, if there is no if statement at all.

=0, if there is `if` but no accompanying `else`.

$$= 1 \text{ (for the accompanying else) } + \text{height(Ladder from this else)}$$

There will be many such ladders, the last line must print the height of the longest ladder present in the program. You should keep track of ladders across functions also. Note that there can be ties also.

if-else max-depth =  $\max\{height(Ladder_1), \dots, height(Ladder_n)\}$  when there are  $n$  ladders in the input file.

- [2] On rejection by the parser, the following should be printed:

```
parsing terminated [syntax error]
```

- [3] On rejection by the lexer when there are unclosed comments, the following should be printed:

```
lexing terminated [lexer error]: ill-formed comment
```

- [4] *On rejection by your a.out file it must handle these cases:*

*Case 1: (invalid number of command-line arguments)-*

## Files

main

Go to file

>  Pro

- `language_processor`

- >  mul\_add\_pred\_swap

```
> playground
```

▼  yapı

```
> testcases
```

 Makefile

 ReadMe.md

 assign\_op.c

 ex1.c

 ex2.c

 lang\_spec.odt

 lang\_spec.pdf

lex.yy.c

 y.tab.c y.tab.h

 `yapl`

yapl.l

yapl.y

 LICENSE

 README.md

CS-327-Compilers / yapl / lang\_spec.pdf

77.7 KB

[↑ Top](#)

```
process terminated [input error]: invalid number of command-line arguments
```

Case 2: (no such file <filename> exists)-

```
process terminated [input error]: no such file <filename> exists
```

**Note:** The Makefile should run `lex`, `yacc`, compile the generated code and generate an executable `a.out` file. Please be careful about the naming conventions and structure of the directory.

### Sample testcase #1

### Input

```
int *var=6;
struct mystruct *ms=&var;

int auto, static, inline;

extern void *k;
int p;
int p;
int p;

int *hh(char *p);

int main(int b)
{
 int auto=2,b=3,c;
 c=auto+b;
 printf("%d",c);

 struct player
 {
 int a;
 double c;
 };

 int *jj=&auto;

 System.out.print("java here");
```

77.7 KB

```

1 int n=7;

 scanf("%d",&n);

 if(n==0)
 printf("%d\n",n+1);
 else if(n==1)
 printf("%d\n",n+2);

 return(NULL);
1

```

### Expected output

```
parsing successful
#global_declarations = 10
#function_definitions = 2
#integer_constants = 20
#pointers_declarations = 12
#ifs_without_else = 2
if-else max-depth = 3
```

### Sample testcase #2

### Input

```
int a, b, c;

int main(int *argc, char* argv[])
{
 int a, i;
 int b[10];
 for(i=1; i<=10; ++i) {
 b[i-1] = i*i; // square
 }
 int *c = b;
 i = 0;
 int d= 0;
 while(i<10) {
 d += *(c + i);
 }
 printf("%d\n", d); // print sum of values
 return 0;
}
```

### Expected output

```
parsing terminated [syntax error]
```



Activities

Google Chrome

Jan 4 5:53 PM

Inbox - shouvik.mondal

Inbox - shouvik.mondal

CS327\_Lecture\_1.pptx

SET-IITGN/CS-327-Compilers

github.com/SET-IITGN/CS-327-Compilers

Q Type to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

SET-IITGN / CS-327-Compilers

Public

Edit Pins

Unwatch 1

Fork 0

Star 2

main 1 Branch 0 Tags

Go to file

Add file

Code

shouvik149

Update README to reflect additional language processor

6f1f0a4 · last week

27 Commits

|                    |                                                        |           |
|--------------------|--------------------------------------------------------|-----------|
| Pro                | Update ReadMe.md                                       | last week |
| language_processor | Add files via upload                                   | last week |
| mul_add_pred_swap  | Clarify language specification statement in ReadMe     | last week |
| playground         | Update ReadMe.md                                       | last week |
| yapl               | Update ReadMe.md                                       | last week |
| LICENSE            | Initial commit                                         | last week |
| README.md          | Update README to reflect additional language processor | last week |

README Apache-2.0 license

CS-327-Compilers

Jan-May 2026, IIT Gandhinagar

Getting Started

You can use the materials (codebase) in this repository to get started for building your own compiler. Four sample language processors (source code + binary) designed by me are available here.

- Pro ( pro ): Scanner, Parser, Analyzer of sentences expressed in Propositional Logic.
- language\_processor ( sm ): Scanner, Parser, Analyzer of sentences expressing arithmetic operations in bash .

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

C 98.9% Other 1.1%

Suggested workflows

Based on your tech stack

32



Files

main

Go to file

Pro

language\_processor

mul\_add\_pred\_swap

Makefile

ReadMe.md

input.txt

lex.yy.c

maps

mul\_add\_pred\_swap.l

mul\_add\_pred\_swap.y

y.tab.c

y.tab.h

playground

yapl

LICENSE

README.md

CS-327-Compilers / mul\_add\_pred\_swap /

ReadMe.md

Context-free grammar (CFG) with the precedence of addition and multiplication swapped. This means + has higher precedence than \*

E -> E \* T | T

T -> T + F | F

F -> NUM

| Operator | Precedence | Associativity |
|----------|------------|---------------|
| +        | Higher     | Left          |
| *        | Lower      | Left          |

Numbers: Arbitrary length via lexer [0-9]+ → single NUM token

Lexical Analyzer (scanner)

%{  
#include <stdlib.h>  
#include "y.tab.h"  
extern char mytext[];  
%}  
  
%%  
  
"+" {return ADD;}  
"\*" {return MUL;}  
[0-9]+ {strcpy(mytext, yytext); return NUM;}  
.|\\n {}  
  
%%  
  
int yywrap(void) {  
return 1;  
}}

Syntax Analyzer (parser)

Context Free Grammar

When a task involves dividing the input into units and establishing some relationship among those units, you should think of Lex and yacc.

part #1: C code (mostly headers)

part #2: "reg-ex patterns" {actions}

part #3: C code (other code)

Output: lex.yy.c containing yylex() //our lexer

yytext contains the text that matched the pattern

Files

main

Go to file

Pro

language\_processor

mul\_add\_pred\_swap

Makefile

ReadMe.md

input.txt

lex.yy.c

maps

mul\_add\_pred\_swap.l

mul\_add\_pred\_swap.y

y.tab.c

y.tab.h

playground

yapl

LICENSE

README.md

CS-327-Compilers / mul\_add\_pred\_swap /

Syntax Analyzer (parser)

lex.yy.c containing yylex() //our lexer entry point

part #1:(tokens, headers, starting non-terminal)

part #2:(CFG productions)

part #3:(main(), syntax error handler, other C code)

yyparse() is our parser entry point

grand entry point for the lexer+parser combination

Output #1: y.tab.c //our parser

Output #2 (opt.): y.tab.h //token definitions

```
}

%token ADD MUL NUM
%{
 #include<stdio.h>
 #include<stdlib.h>
 void yyerror(char *);
 int yylex(void);
 char mytext[100];
 extern char *yytext;
%}
%start S

%%

S: E {$$=$1;printf("%d\n",$$);}
;

E: E MUL T {$$=$1*$3;}
 | T {$$=$1;}
;

T: T ADD F {$$=$1+$3;}
 | F {$$=$1;}
;

F: NUM {$$=atoi(yytext);}
;

%%

void yyerror(char *s) {
 fprintf(stderr, "%s\n", s);
}

int main(void) {
 yyparse();
 return 0;
}
```

- gcc
- yacc
- lex



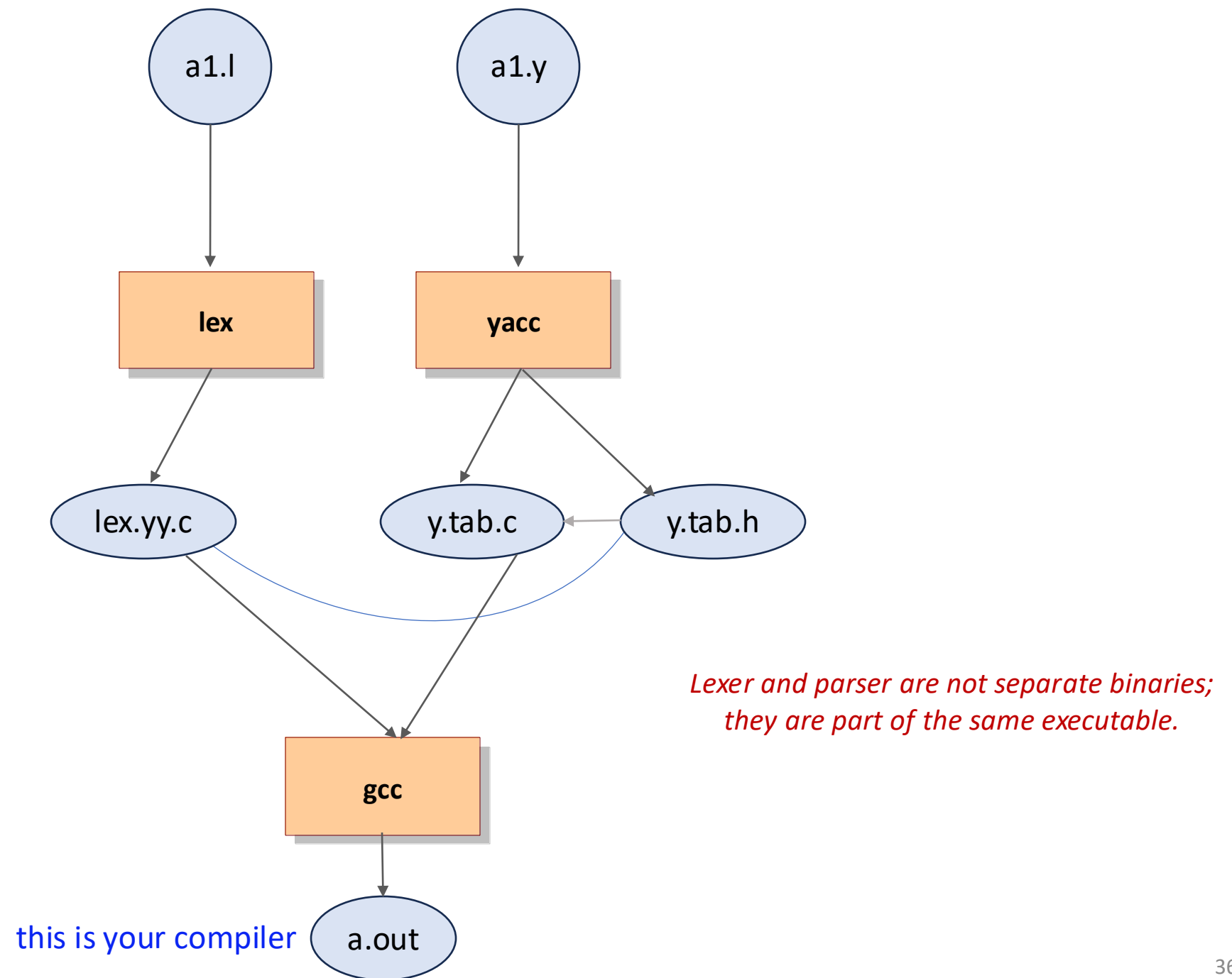
10

build compiler

execute compiler

□

10



Google Chrome

Inbox - shouvick.mondal

Inbox - shouvick.mondal

CS327\_Lecture\_1.pptx

SET-IITGN/CS-327-Comp

CS-327-Compilers/Pro a

CS-327-Compilers/langu

CS-327-Compilers/mul\_

github.com/SET-IITGN/CS-327-Compilers/tree/main/mul\_add\_pred\_swap#context-free-grammar-cfg-with-the-precedence-of-addition-and-multiplication-swapped-this-means-has-higher-precedence-than-

Files

main

+

Q

Go to file

> Pro

> language\_processor

> **mul\_add\_pred\_swap**

Makefile

ReadMe.md

input.txt

lex.yy.c

maps

mul\_add\_pred\_swap.l

mul\_add\_pred\_swap.y

y.tab.c

y.tab.h

> playground

> yapl

LICENSE

README.md

CS-327-Compilers / mul\_add\_pred\_swap /

3+

syntax error

\$ ./maps

4+9\*(7)

91

\$ ./maps

(8((

8

\$ ./maps

int a=8;

int b=a+8\*2;

printf("%d", b);

32

\$ ./maps

a=8;

kuch bhi;

+5\*iitgn

4

52

Observation

The way you specify a programming language (syntax + semantics) determines what you deploy through software written in that language, including bugs (features).

37

# Texts, References, and Acknowledgements

1. Alfred Aho, Monica Lam, Ravi Sethi, Jeff Ullman. [Compilers: Principles, Techniques, and Tools](#), Pearson Education India; 2nd Edition, 2013.
2. Steven Muchnick, [Advanced Compiler Design and Implementation](#), Morgan Kaufmann; 1st edition 1997.
3. John R. Levine, Tony Mason, and Doug Brown, [Lex & Yacc](#) (2nd ed.). O'Reilly & Associates, Inc., USA, 1992.
4. Jonathan Bartlett, [Programming from the Ground Up](#), 2003.
5. Wikibooks, [x86 Assembly](#), [GNU assembly syntax](#) (online).